Clases

Aunque podemos emplear las funciones constructoras para crear nuevos objetos, en Javascript disponemos ahora de la **posibilidad de crear clases** más apropiadas para trabajar con objetos. La sintaxis es la siguiente:

```
class Clase {
    propiedades;
    constructor(){…}
    métodos(){…}
}
```

En realidad, **la clase se convierte a una función a la que se asignan el constructor y los métodos como propiedades del prototipo**.

No puede emplearse sin el operador 'new'.

Los **métodos** de la clase **no son enumerables** (se asigna el `false` al atributo `enumerable` de todos los métodos de la propiedad `prototype`) y las clases **siempre se usan en modo estricto**.

```
class Persona {
    set nombre(nombre){
        this._nombre_ = nombre;
    }
    get nombre(){
        return this._nombre_;
    }
    edad = 18;
    constructor(nombre){
        this.nombre = nombre;
    }
    hola(){
        return `Hola. Soy ${this.nombre} y tengo ${this.edad} años`;
    }
}
// A continuación vamos a crear un objeto de tipo persona(Objeto: la instancia de una clase)
let p = new Persona("Juan");
console.log(p.hola());
> Hola. Soy Juan y tengo 18 años
let p2 = new Persona();
```

```
console.log(p2.hola());

>Hola. Soy undefined y tengo 18 años

p2.nombre = "Ana"

p2.edad = 23;

console.log(p2.hola());

>Hola. Soy Ana y tengo 23 años
```

NOTA: Todos los métodos de la clase se almacenan en el prototipo de la misma de modo que sólo habrá una para todas las instancias de la clase.

NOTA: Si especificamos una propiedad o un método como static se almacenarán en la clase y no en el prototipo. La diferencia es que hay que invocarlos a través de la clase (de hecho da error intentar hacerlo a través de una instancia de la misma).

```
class Animal {

    static tipo = "Animal";

    constructor(nombre) {

        this.nombre = nombre;

    }

    set nombre(nombre){

        this._nombre = nombre;

    }

    get nombre(){

        return this._nombre;

    }

    saludo(){

        console.log(`Soy un ${this.nombre}`);

    }

    static adios(){

        console.log("Adiós " + Animal.tipo);

    }

}

perro = new Animal("perro");

perro.saludo()

>Soy un perro

perro.adios()

>Uncaught TypeError: perro.adios is not a function

// no funciona porque la funcion adios es estática. Es decir, es de la clase (Animal) y no de la instancia perro

at <anonymous>:1:7
```

```
Animal.adios()

>Adiós animal

/* console.dir() Es la manera de ver todas las propiedades de un objeto JavaScript especificado por consola,

 mediante la cual el desarrollador puede facilmente obtener las propiedades del objeto.*/


console.dir(Animal)

class Animal
  tipo: "Animal"
  adios: ƒ adios()
  arguments: (...)
  caller: (...)
  length: 1
  name: "Animal"
  prototype: {constructor: ƒ, saludo: ƒ}
  proto__: f()
```

Dentro de un método estático **this** apunta al constructor.

Herencia de Clases

De manera similar a Java, en Javascript disponemos de la palabra clave `extends` para **implementar la herencia**.

```
class Persona {

    set nombre(nombre){

        this._nombre_ = nombre;

    }

    get nombre(){

        return this._nombre_;

    }

    edad = 18;

    constructor(nombre){

        this.nombre = nombre;

    }

    hola(){

        return `Hola. Soy

    ${this.nombre} y tengo

    ${this.edad} años`;

    }

}
```

```
class Alumno extends Persona {

    set curso(curso){

        this._curso = curso;

    }

    get curso(){

        return this._curso;

    }

}

let juan = new Alumno("Juan")

juan.hola()

>"Hola. Soy Juan y tengo 18 años"
```

De nuevo, de manera similar a Java podemos sobreescribir los métodos de la clase padre **e incluso usar** `super()` para **llamar a un constructor** de la clase padre (sólo desde el constructor de la clase hija) y `super.metodo()` para llamar a un método de la misma.

Si **no proporcionamos ningún constructor** a una clase, **Javascript añade uno por defecto** con el siguiente código:

```
constructor(…args){

    super(…args);

}
```

En nuestro ejemplo, el constructor proporcionado por Javascript llama al constructor padre que recibe un parámetro y lo ejecuta.

NOTA: En Javascript **sólo puede haber un constructor** por cada clase (se presenta ejemplo Clase1 con el error) y, **si hay herencia, es obligatorio** llamar al método `super()` de la clase padre.(error que se vé en la clase Persona2, mediante la ausencia de la llamada al super()

```
class Clase1 {

    constructor(){}

    constructor(nombre){}

}

> Uncaught SyntaxError: A class may only have one constructor
```

```
class Persona2 extends Persona{

    constructor(){}

}
```

```
let pp = new Persona2();

>Uncaught ReferenceError: Must call super constructor in derived class before accessing 'this' or returning from derived constructor

at new Persona2 (<anonymous>:2:16)

at <anonymous>:1:10
```

NOTA: Los **método estáticos también se heredan**:

```
class Perro extends Animal{

    constructor(){

        super("perro");
}


let elvis = new Perro()

elvis
>Perro {_nombre: "perro"}

elvis.saludo()
>Soy un perro

Perro.adios()

>Adiós Animal
```

Acceso a los métodos

En Javascript, como ya sabemos, **todas las propiedades y los métodos son públicos**. Aún así , hay una norma que indica que las **variables que empiezan por _ no debieran ser accedidas directamente**, sino a través de los métodos de acceso. Dado que estas variables y métodos son heredables, estaríamos hablando de propiedades y métodos protegidos.

. Esto era una convención o norma en la comunidad, pero podríamos seguir accediendo desde fuera a dicha variable. Como novedad con el símbolo `#`. se pueden definir variables privadas reales. Para más información acceder a:

https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Classes/Private_class_fields